SignalLab

Signal generator and digital filter designer built with Python and Streamlit

1. Overview

SignalLab is an interactive tool for learning digital signal processing (DSP). You can generate a test signal or upload a WAV file, add noise at a chosen signal-to-noise ratio (SNR), design a digital filter, and see the result in the time domain, the frequency domain, and as a spectrogram. The filtered audio can be played and downloaded.

2. Features

- **Signals:** sine, square, sawtooth, chirp and multi-tone, or an uploaded WAV file.
- **Noise:** white Gaussian noise at any SNR from 0 to 40 dB, with SNR measured before and after filtering.
- **Filters:** lowpass, highpass, bandpass and bandstop, designed as FIR (window), Butterworth, Chebyshev I or Elliptic.
- **Safety checks:** cutoff frequencies are validated against the Nyquist limit and unstable filters are detected.
- **Zero-phase option:** forward-backward filtering (filtfilt) to avoid phase distortion.
- **Plots:** time domain, magnitude spectrum, filter frequency response and spectrogram.
- **Audio:** built-in player and a download button for the filtered WAV file.

3. Tech stack

Library	Used for
NumPy	Array maths, FFT, noise generation
SciPy	Filter design (firwin, butter, cheby1, ellip), filtering, spectrogram, WAV I/O
Matplotlib	The four plots
Streamlit	The web interface (sliders, dropdowns, buttons)
pytest	Automated tests of the DSP code

4. Project structure

File	Purpose
dsp.py	Core DSP functions with no user-interface code, so they are easy to test
app.py	Streamlit interface: sidebar controls, processing and plots
test_dsp.py	13 pytest tests
requirements.txt	Python dependencies

5. How to run

```
pip install -r requirements.txt
python -m streamlit run app.py
python -m pytest
```

The app opens in the browser at `http://localhost:8501`. Use `python -m` if the `streamlit` command is not found on Windows.

6. How it works

The processing pipeline has four steps:

- **Generate:** `generate_signal()` builds the clean signal at the chosen sampling rate.
- **Add noise:** `add_noise()` scales Gaussian noise so the result has the requested SNR.
- **Filter:** `design_filter()` returns a filter object whose `apply()` method cleans the signal.
- **Analyse:** `spectrum()`, `get_spectrogram()` and `snr_db()` measure the result.

7. Example run

Setting	Value
Signal	Multi-tone, base frequency 860 Hz (tones at 860, 2150 and 3440 Hz)
Sampling rate / duration	16000 Hz / 2.9 s
Noise	SNR = 29 dB
Filter	Lowpass, FIR (window), cutoff 1200 Hz, order 4, zero-phase

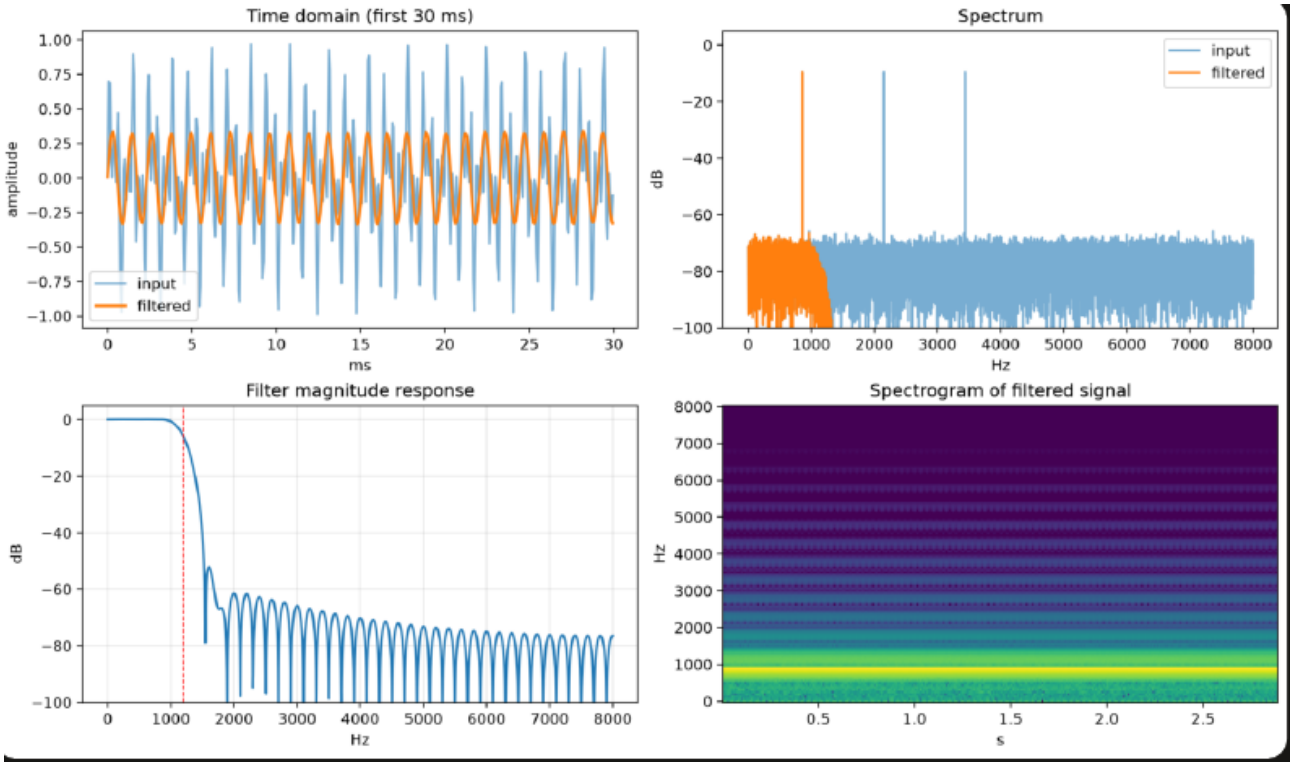


Figure 1: Time signal, spectrum, filter response and spectrogram for the example run.

8. Understanding the results

In the example the app reports an SNR of 29.0 dB before filtering and 1.8 dB after. This looks like the filter made the signal worse, but it is the expected result for this setup:

- The spectrum shows the lowpass filter removed the noise above 1200 Hz and kept the 860 Hz tone, as designed.
- The 2150 Hz and 3440 Hz tones are also part of the clean signal, and the 1200 Hz cutoff removes them.

- SNR is measured against the full clean signal, so the removed tones count as error. The three tones have equal power, so the filter keeps 1 part and loses 2. Signal power divided by error power is 3 to 2, which is 10 x log10(1.5), about 1.8 dB. This matches the number the app shows.
- In the spectrum, the orange (filtered) curve keeps only the 860 Hz peak, confirming this.

To see the SNR improve, use a single sine wave, or choose a cutoff above the highest tone (for example 4000 Hz for this multi-tone signal) so the filter removes only noise.

9. Testing

The 13 pytest tests check that every filter type and method passes and blocks the right frequencies, that filtering improves SNR for a sine wave in noise, that the added noise has the requested SNR, that the FFT peak is at the correct frequency, that invalid inputs are rejected, and that WAV files survive a read/write round trip.

10. Future work

- Pole-zero plot and group delay display.
- Live microphone input for real-time filtering.
- 50 Hz hum-removal preset and an ECG denoising demo.
- Adaptive (LMS) filter for noise cancellation.
- Public deployment on Streamlit Community Cloud.

11. Development note

This project was built with the help of an AI assistant. The author ran, tested and documented it, and is extending it with further features.